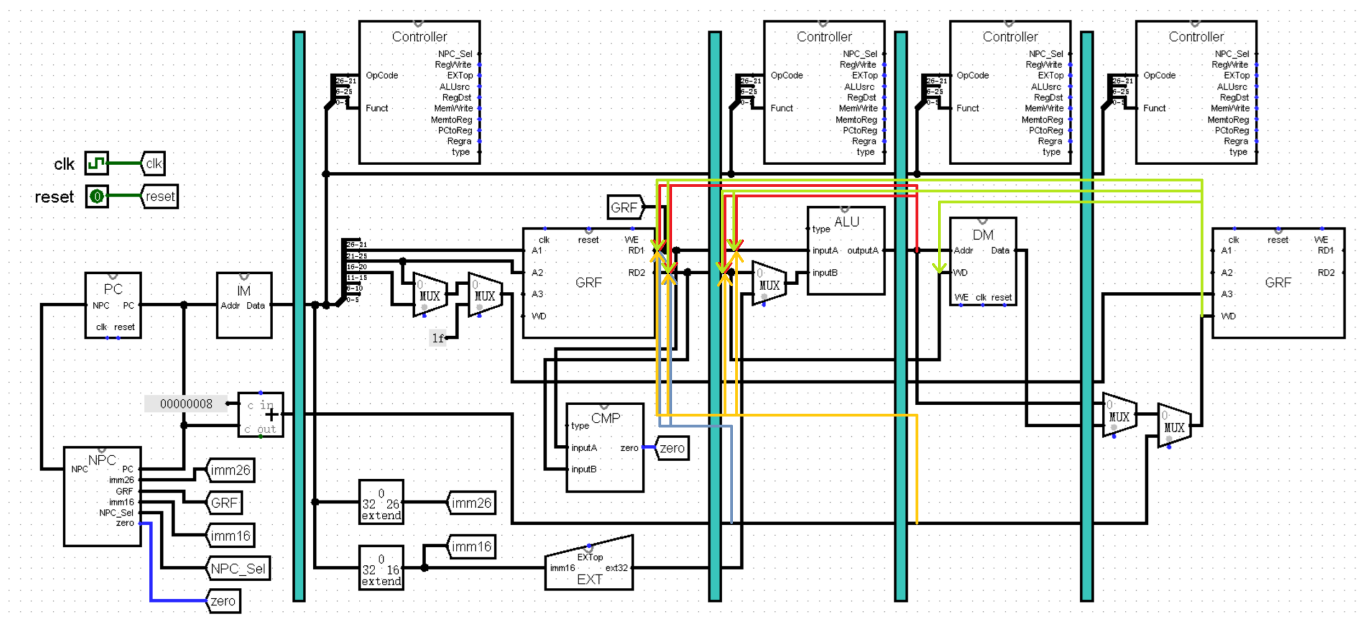
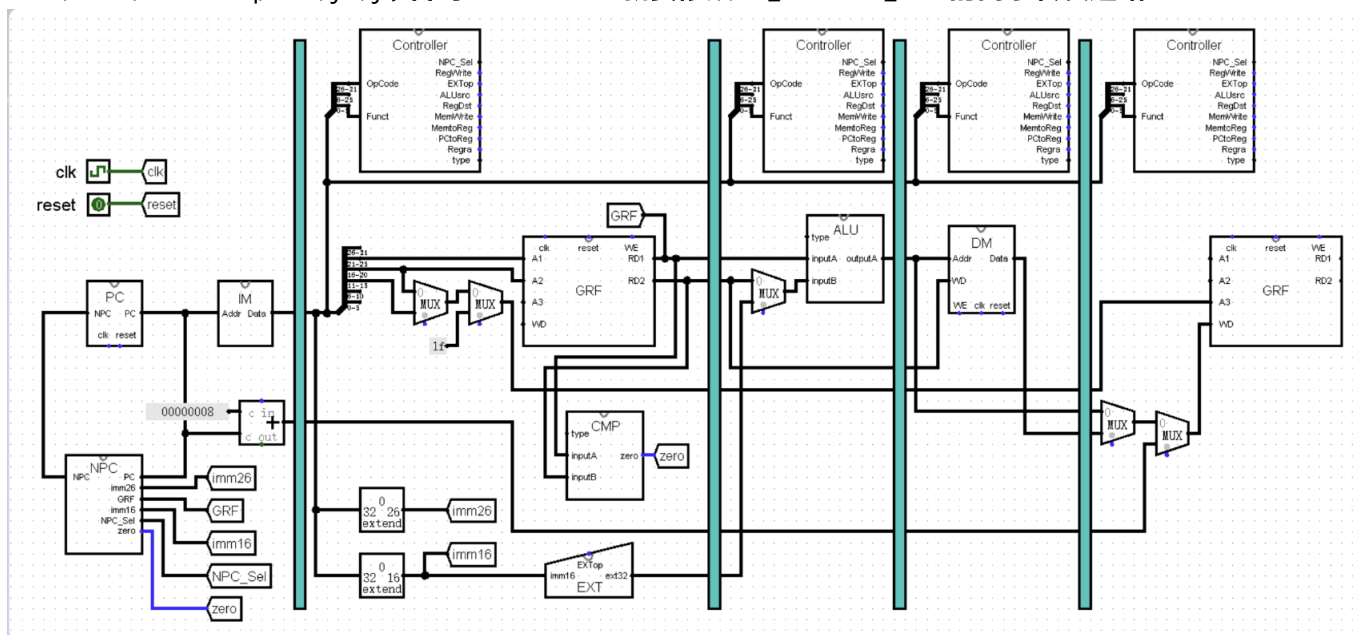
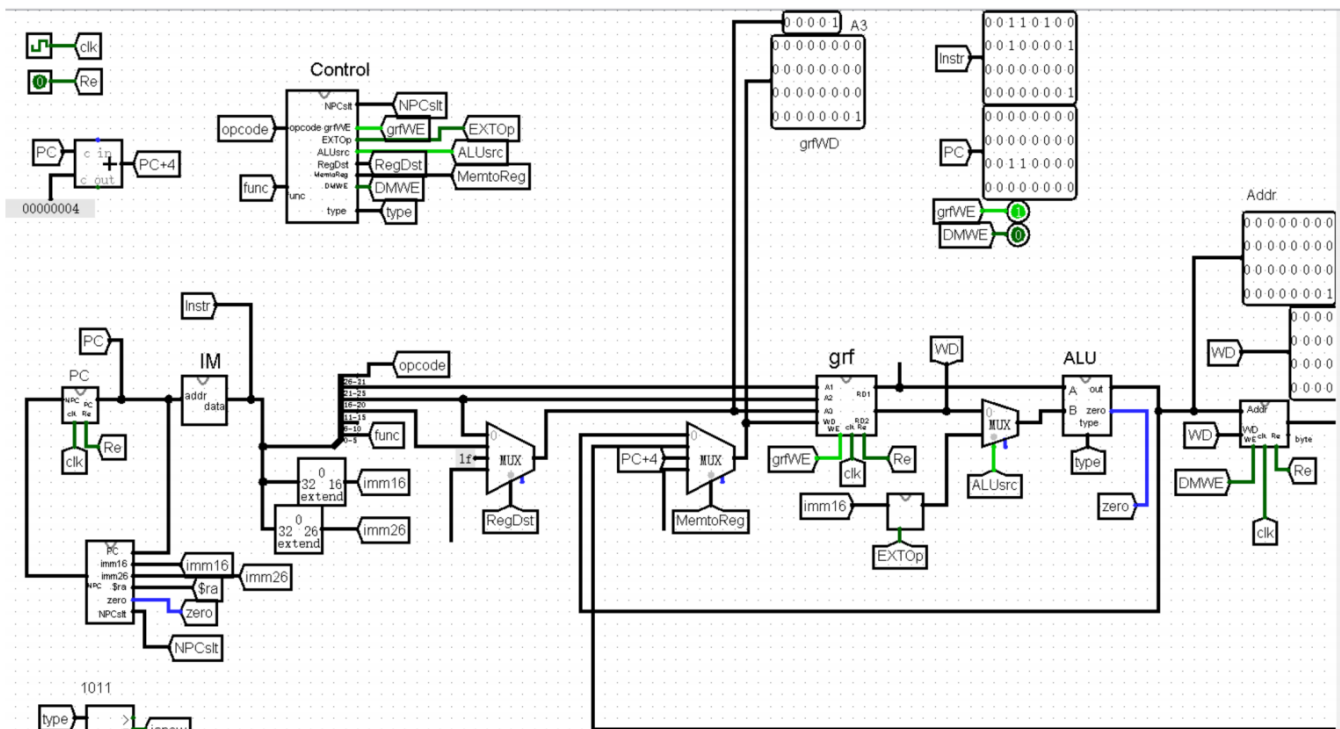


实现的指令集: {add, sub, and, or, slt, sltu, lui, addi, andi, ori, lb, lh, lw, sb, sh, sw, mult, multu, div, divu, mfhi, mflo, mthi, mtlo, beq, bne, jal, jr} **看到\$31<-PC+8 就要修改isE PC8 isM PC8的两条转发通路!!!**



p4cpu:



$$temp \leftarrow (HI \parallel LO) - (GPR[rs] * GPR[rt])$$

$$HI \leftarrow temp_{63..32}$$

$$LO \leftarrow temp_{31..0}$$

解决方法:  $t = \{HI, LO\} + \$signed(\{32\{A[31]\}\}, A[31:0]) * \$signed(\{32\{B[31]\}\}, B[31:0])$ ;

```
.text
ori $2,$2,-1
ori $3,$3,1
ori $4,$4,0xffff
sw $4,0($0)
mfhi $10
mflo $11
```

## 阻塞过度导致的TLE问题

在DE\_stall中分成isnew lisnew两种情况:

```
assign DE_rs_Stall =(E_type==4'b1011&&D_rs!=5'b0&& 这个约束条件根据题目调整D_rs >=
E_A3这个约束条件根据题目调整 && D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse <
E_t_new)
|| (E_type!=4'b1011 (直接加这一句就可以了) &&E_A3 != 5'b0 && D_rs == E_A3 &&
D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse < E_t_new);
```

```
// 如果 E 级指令是 lwtxr 指令，则当 D_rs 不是 0 号寄存器，且 D_t_rsuse 存在，且 D_rs
大于等于 E_rt（即 E_A3 ）时才阻塞
// 需要注意在 E 级指令不是 lwtxr 指令时，在判断条件前加上 E_type != LWTXR ，避免被上一
条判断漏下来的 lwtxr 指令进入接下来的判断！
// 如果 M 级指令是 lwtxr 指令，需要使用修正过后的 A3 值
assign D_stall = (E_type == 4'b1011 && D_rs != 5'b00000 && D_t_rsuse != 4'hf &&
D_rs >= E_A3) ? 1'b1 :
    (E_type == 4'b1011 && D_rt != 5'b00000 && D_t_rtuse != 4'hf &&
D_rt >= E_A3) ? 1'b1 :
    (E_type != 4'b1011 && E_A3 != 5'b00000 && D_rs == E_A3 &&
D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse < E_t_new) ? 1'b1 :
    (E_type != 4'b1011 && E_A3 != 5'b00000 && D_rt == E_A3 &&
D_t_rtuse != 4'hf && E_t_new != 4'hf && D_t_rtuse < E_t_new) ? 1'b1 :
    (M_fixedA3 != 5'b00000 && D_rs == M_fixedA3 && D_t_rsuse != 4'hf
&& M_t_new != 4'hf && D_t_rsuse < M_t_new) ? 1'b1 :
    (M_fixedA3 != 5'b00000 && D_rt == M_fixedA3 && D_t_rtuse != 4'hf
&& M_t_new != 4'hf && D_t_rtuse < M_t_new) ? 1'b1 :
    (W_A3 != 5'b00000 && D_rs == W_A3 && D_t_rsuse != 4'hf && W_t_new
!= 4'hf && D_t_rsuse < W_t_new) ? 1'b1 :
    (W_A3 != 5'b00000 && D_rt == W_A3 && D_t_rtuse != 4'hf && W_t_new
!= 4'hf && D_t_rtuse < W_t_new) ? 1'b1 : 1'b0;
```

总述

CPU 为的 5 级流水线阶段为：

| 阶段              | 简称 | 功能概述                        |
|-----------------|----|-----------------------------|
| 取指阶段（Fetch）     | F  | 从指令存储器中读取指令                 |
| 译码阶段（Decode）    | D  | 从寄存器文件中读取源操作数并对指令译码以便得到控制信号 |
| 执行阶段（Execute）   | E  | 使用 ALU 执行计算                 |
| 存储阶段（Memory）    | M  | 读或写数据存储器                    |
| 写回阶段（Writeback） | W  | 将结果写回到寄存器文件                 |

模块名

M级

DE 针对lw lh lb

| 信号名  | 方向 | 描述                                          |
|------|----|---------------------------------------------|
| addr | I  | 加载操作的地址输入，其最低0 1位用于LB（字节加载）/LH（半字加载）的半字位置定位 |

| 信号名        | 方向 | 描述                                            |
|------------|----|-----------------------------------------------|
| data       | I  | 从存储器读取的原始32位数据，作为LB/LH/LW加载操作的数据源             |
| type       | I  | 加载类型控制信号（LB=5'b01011、LH=5'b01100、LW=5'b01101） |
| fixed_data | O  | 经过符号扩展的最终加载数据 <b>符号扩展</b> 为32位                |

BE 针对sw sh sb

| 信号名        | 方向 | 描述                                                               |
|------------|----|------------------------------------------------------------------|
| addr       | I  | 存储写操作的地址输入，其最低1位（SH）/最低2位（SB）用于定位半字/字节的存储位置                      |
| data       | I  | 待写入存储器的原始32位数据                                                   |
| type       | I  | 存储类型控制信号（SB=5'b01110、SH=5'b01111、SW=5'b10000）                    |
| DMWE       | O  | 存储器字节写使能信号（4位，每1位对应1个字节的写使能）                                     |
| fixed_data | O  | 地址对齐后的待写存储器数据：SW直接输出原始数据；SH/SB根据地址将目标半字/字节放到对应字节位置，其余位 <b>补0</b> |

F级

PC

| 信号名       | 方向 | 描述       |
|-----------|----|----------|
| clk       | I  | 时钟信号     |
| reset     | I  | 异步复位信号   |
| PC_en     | I  | PC使能信号   |
| NPC[31:0] | I  | 下一条指令的地址 |
| PC[31:0]  | O  | 目前的指令地址  |

实现方法：用12位地址、32位数据的ROM作为IM。将PC-0x3000存入储存PC的寄存器中，寄存器中的值可以直接对应ROM要获取指令的地址，从而完成指令输出。

NPC

| 信号名         | 方向 | 描述                                 |
|-------------|----|------------------------------------|
| F_PC[31:0]  | 输入 | 当前取指阶段的PC值（指令地址）                   |
| imm16[15:0] | 输入 | 16位立即数（BEQ指令的地址偏移量，需符号扩展后计算跳转地址）   |
| imm26[25:0] | 输入 | 26位立即数（J/JAL指令的跳转地址段，拼接PC高位生成目标地址） |
| GRF[31:0]   | 输入 | 通用寄存器堆读出的地址（JR指令的跳转目标PC值）          |

| 信号名         | 方向 | 描述                                |
|-------------|----|-----------------------------------|
| NPCslt[1:0] | 输入 | 下一条PC选择控制信号（来自D_Control模块，控制跳转逻辑） |
| zero        | 输入 | 比较结果标志（来自CMP模块，BEQ指令跳转的判断条件）      |
| NPC[31:0]   | 输出 | 下一条指令的PC地址（生成PC+4、分支跳转、直接跳转）      |

D级

D\_Control

| 信号名           | 方向 | 描述                             |
|---------------|----|--------------------------------|
| opcode[5:0]   | 输入 | 指令操作码                          |
| func[5:0]     | 输入 | 功能码，用于R型指令细分                   |
| type[3:0]     | 输出 | 运算/操作类型选择信号                    |
| NPCslt[1:0]   | 输出 | 下一条PC选择信号，控制PC的更新方式            |
| ALUsrc        | 输出 | ALU操作数B的来源选择（寄存器或立即数扩展）        |
| grfWE         | 输出 | 通用寄存器堆写使能信号                    |
| RegDst[1:0]   | 输出 | 通用寄存器堆写地址选择（rd、rt或31号）         |
| EXTOp         | 输出 | 立即数扩展类型选择（符号扩展或零扩展）            |
| MemtoReg[1:0] | 输出 | 寄存器堆写入数据来源选择（ALU结果、内存数据或PC+8等） |
| DMWE          | 输出 | 数据存储器写使能信号                     |
| t_rs[3:0]     | 输出 | rs寄存器被使用的时间                    |
| t_rt[3:0]     | 输出 | rt寄存器被使用的时间                    |
| t[3:0]        | 输出 | 产生新数据的时间                       |

CMP

| 信号名          | 方向 | 描述                                  |
|--------------|----|-------------------------------------|
| inputA[31:0] | 输入 | 比较操作数A                              |
| inputB[31:0] | 输入 | 比较操作数B                              |
| type[3:0]    | 输入 | 比较类型                                |
| zero[0]      | 输出 | 比较结果标志：BEQ类型下inputA等于inputB时为1，否则为0 |

EXT

| 信号名         | 方向 | 描述            |
|-------------|----|---------------|
| imm16[15:0] | 输入 | 16位立即数（待扩展数据） |

| 信号名         | 方向 | 描述                     |
|-------------|----|------------------------|
| EXTOp       | 输入 | 扩展模式选择信号（0=零扩展，1=符号扩展） |
| EXT32[31:0] | 输出 | 扩展后的32位数据              |

grf

| 信号名         | 方向 | 描述                  |
|-------------|----|---------------------|
| clk         | I  | 时钟信号                |
| reset       | I  | 异步复位信号              |
| grfWE       | I  | 写入使能信号              |
| PC[31:0]    | I  | 当前PC值--W级的PC(W_PC)  |
| A1[4:0]     | I  | 指定一个寄存器并将其中数据读出到RD1 |
| A2[4:0]     | I  | 指定一个寄存器并将其中数据读出到RD2 |
| A3[4:0]     | I  | 指定一个寄存器作为写入数据的目标    |
| grfWD[31:0] | I  | 要写入的数据              |
| RD1[31:0]   | O  | \$rs                |
| RD2[31:0]   | O  | \$rt                |

该模块直接使用P0中搭建的GRF(并新增PC的输入便于输出测试)。

E级

E\_Control

ALU

| 信号名       | 方向 | 描述       |
|-----------|----|----------|
| A[31:0]   | 输入 | 操作数A     |
| B[31:0]   | 输入 | 操作数B     |
| type[3:0] | 输入 | 操作类型选择信号 |
| out[31:0] | 输出 | 运算结果     |

思考题

1. 为什么需要有单独的乘法部件而不是整合进ALU？为何需要有独立的HI、LO寄存器？

乘法除法运算时间较长，与ALU分开可以使得ALU并行进行运算，加快处理效率。独立的HI LO寄存器同样方便时间较长的运算直接存储，不需要使整个CPU等待

## 2. 真实的流水线CPU是如何使用实现乘除法的？请查阅相关资料进行简单说明。

乘法除法通过加法移位，每个周期计算若干位的结果，在一定周期后得出结果。除法也是通过迁移求部分和，累减实现，对齐余数和除数后再相减，一个周期完成若干位计算即可。

## 3. 请结合自己的实验分析，你是如何处理Busy信号带来的周期阻塞的？

当 D 级指令是 8 条乘除法指令，且 start 信号或 busy 信号为 1 时，直接阻塞

## 4. 请问采用字节使能信号的方式处理写指令有什么好处？（提示：从清晰性、统一性角度考虑）

字节使能信号可以根据1/0情况直接确定写入内存的位置，同时不论何种存储都可以通过字节使能信号进行控制，控制方法统一，且更容易和mips\_tb衔接从而进行测试

## 5.请思考，我们在按字节读和按字节写时，实际从 DM 获得的数据和向 DM 写入的数据是否是一字节？在什么情况下我们按字节读和按字节写的效率会高于按字读和按字写呢？

在实现了字节使能信号之后，我们和DM进行的数据交流均以字为基本单位，而非一字节，DM在获取使能信号后还需要进一步对该信号的解析，这反而降低了按字节读写的效率，当不支持按字写时，按字节写的效率要高于按字写。当我们只需要对字节或半字访问时，按字节访问内存性能更由优势。如果此时还采用按字访问，则需要首先将整个字从内存中拿出来，然后再从字中寻找，效率会更低。

## 6. 为了对抗复杂性你采取了哪些抽象和规范手段？这些手段在译码和处理数据冲突的时候有什么样的特点与帮助？

尽量不在主模块上进行操作，为控制信号留出充足的位宽，减少无效控制信号的数量，这样每个模块只需要根据控制信号按照自己的方式进行处理，根据AT相关数值判断数据冲突。

## 7. 在本实验中你遇到了哪些不同指令组合产生的“冲突”？你又是如何解决的？相应的测试样例是什么样的？

与P5相比，仅仅增加了两种冲突，一种是乘除法内部冲突，即乘除模块还未运算出结果并写进hi与lo，便使用mfhi指令调用了寄存器。另一种是mflo与mfhi还未将值存进寄存器就需要使用相关寄存器的值。对于第一种冲突，在Busy信号为1且需要使用乘除法模块时阻塞。对于第二种冲突，将T\_new置为1并进行转发，测试样例如下：

```
ori $1,$0,20
add $2,$0,35
mult $1,$2
mflo $3
mfhi $4
div $2,$1
add $5,$3,$4
```

## 8.如果你是手动构造的样例，请说明构造策略，说明你的测试程序如何保证覆盖了所有需要测试的情况；如果你是完全随机生成的测试样例，请思考完全随机的测试程序有何不足之处；如

果你在生成测试样例时采用了特殊的策略，比如构造连续数据冒险序列，请你描述一下你使用的策略如何结合了随机性达到强测的效果。

本次测试采用“半随机自动生成 + 手动定向优化”的混合策略，尽力全面验证处理器指令执行的正确性，采用Python随机生成（详见Student\_judge）的方法，覆盖功能型指令的逻辑完整性、跳转指令的控制流稳定性。在基础模板的基础上，通过手动修改指令序列，针对性引入数据冒险与控制冒险场景两类指令在数据冒险、控制冒险场景下的处理能力，既保证测试用例的批量生成效率，又精准覆盖高风险的边缘执行场景。

我的测试代码：

```
ori $0, $0, 100
ori $1, $0, 48
ori $2, $0, 12
ori $3, $0, 52
ori $4, $0, 144
ori $5, $0, 68
ori $6, $0, 152
ori $7, $0, 48
ori $8, $0, 100
ori $9, $0, 148
ori $10, $0, 80
ori $11, $0, 60
ori $12, $0, 100
ori $13, $0, 24
ori $14, $0, 72
ori $15, $0, 92
ori $16, $0, 44
ori $17, $0, 140
ori $18, $0, 148
ori $19, $0, 24
ori $20, $0, 100
ori $21, $0, 108
ori $22, $0, 72
ori $23, $0, 8
ori $24, $0, 144
ori $25, $0, 84
ori $26, $0, 140
ori $27, $0, 16
ori $28, $0, 116
ori $29, $0, 124
ori $30, $0, 64
ori $31, $0, 148

add $1,$2,$3
ori $3,$4,108
ori $0,$2,80
jal Test1
ori $0,$1,0
Back1:
sub $1,$1,$1
sw $1,148($1)
sw $1,112($0)
```



```
lw $2,116($0)
jal Test2
lw $0,144($0)
Back2:
sub $4,$4,$4
sw $4,132($4)
sub $4,$4,$4
ori $4,$4,68
sw $3,-16($4)
sw $0,116($0)
jal Test3
sw $3,16($0)
Back3:
sub $3,$2,$1
lw $0,136($0)
lui $2,112
jal Test4
sub $2,$2,$1
Back4:
sw $1,32($0)
sub $1,$1,$1
lw $2,88($1)
sub $1,$1,$2
jal Test5
lui $0,92
Back5:
sw $1,20($0)
sub $4,$0,$4
sw $0,116($0)
jal Test6
add $3,$1,$3
Back6:
add $0,$1,$3
lui $3,136
sw $2,8($0)
jal Test7
sub $2,$4,$1
Back7:
sub $1,$1,$1
ori $1,$1,80
sw $1,-76($1)
lw $3,100($0)
lui $2,32
jal Test8
sw $4,104($0)
Back8:
sw $0,44($0)
sw $4,60($0)
sub $0,$0,$1
jal Test9
sub $4,$1,$2
Back9:
lui $2,148
sub $2,$2,$2
```

```
ori $2,$2,4
lw $1,0($2)
lw $1,84($0)
jal Test10
sw $4,92($0)
Back10:
lw $0,40($0)
sub $4,$4,$4
sw $0,60($4)
sw $1,64($0)
jal Test11
lw $1,96($0)
Back11:
ori $2,$1,124
sub $4,$4,$4
lw $1,48($4)
lui $3,84
jal Test12
lw $1,4($0)
Back12:
sub $2,$2,$2
ori $2,$2,128
lw $3,-88($2)
sw $1,60($0)
lw $3,28($0)
jal Test13
sw $0,84($0)
Back13:
lw $3,8($0)
add $3,$1,$4
sub $0,$0,$0
sw $4,96($0)
jal Test14
sub $1,$0,$3
Back14:
add $3,$3,$3
lui $3,0
lw $1,64($0)
jal Test15
ori $0,$3,76
jal End
add $4,$0,$1
```

```
Test1:
add $3,$4,$2
add $1,$2,$1
lui $1,0
sub $4,$4,$4
sw $4,72($4)
lui $2,4
sub $1,$1,$1
lw $0,12($1)
add $2,$4,$1
lw $3,24($0)
```

```
sub $4,$0,$3
sw $2,88($0)
sub $3,$0,$3
sub $4,$3,$0
sub $4,$4,$4
ori $4,$4,152
sw $3,0($4)
sub $4,$4,$4
ori $4,$4,100
sw $1,-8($4)
beq $1,$3,EndTest1
sub $1,$1,$1
beq $0,$2,EndTest1
sub $0,$4,$4
ori $1,$2,116
lw $0,24($0)
add $1,$2,$1
beq $0,$4,EndTest1
sw $0,36($0)
sub $4,$4,$4
ori $4,$4,132
lw $3,-96($4)
add $1,$3,$2
beq $1,$3,EndTest1
sub $1,$0,$1
beq $2,$0,EndTest1
lw $3,128($0)
lui $4,136
EndTest1:
jr $ra
add $0,$2,$4
```

```
Test2:
sub $5,$5,$5
sub $26,$26,$26
ori $26,$26,10
sub $15,$15,$15
ori $15,$15,1
SubTest2:
add $5,$5,$15
beq $5,$26,EndTest2
add $4,$1,$2
sub $1,$1,$1
lui $1,120
lw $0,0($0)
sub $1,$1,$1
ori $1,$1,64
lw $0,-4($1)
lui $1,44
lw $2,116($0)
ori $1,$1,96
lui $1,84
sub $2,$2,$2
```

```
lw $3,112($2)
lw $1,0($0)
sw $1,88($0)
ori $4,$3,84
beq $3,$0,SubTest2
sw $1,48($0)
lw $4,84($0)
lw $3,60($0)
sub $1,$1,$1
ori $1,$1,152
sw $4,-36($1)
sub $4,$4,$2
sub $1,$1,$1
ori $1,$1,56
lw $2,-8($1)
lw $0,60($0)
sw $3,24($0)
sub $2,$2,$2
ori $2,$2,148
lw $2,-44($2)
ori $3,$3,0
sw $2,72($0)
sub $1,$1,$1
ori $1,$1,152
sw $3,-48($1)
sub $4,$4,$4
lw $4,124($4)
EndTest2:
jr $ra
lw $4,152($0)
```

```
Test3:
sub $1,$1,$1
ori $1,$1,132
lw $0,-72($1)
sw $0,108($0)
sw $3,116($0)
lui $0,148
add $0,$4,$4
ori $1,$2,84
add $4,$3,$2
sub $2,$4,$0
sw $0,44($0)
sub $3,$3,$3
lw $0,148($3)
sub $0,$0,$0
sw $4,4($0)
sub $1,$4,$1
sub $3,$1,$1
ori $3,$4,128
lw $3,148($0)
ori $1,$3,44
sub $1,$1,$1
```

```
ori $1,$1,92
lw $4,-40($1)
add $4,$0,$1
sub $4,$4,$4
ori $4,$4,140
lw $4,-64($4)
ori $0,$2,116
sw $0,80($0)
sub $4,$2,$0
sub $3,$3,$3
lw $4,112($3)
add $3,$4,$4
ori $3,$0,56
EndTest3:
jr $ra
lui $3,116
```

```
Test4:
sub $10,$10,$10
sub $9,$9,$9
ori $9,$9,10
sub $14,$14,$14
ori $14,$14,1
SubTest4:
add $10,$10,$14
beq $10,$9,EndTest4
sw $2,144($0)
lw $4,20($0)
sub $2,$0,$3
ori $3,$1,104
lw $0,36($0)
lui $1,96
lw $2,136($0)
add $1,$2,$3
beq $1,$2,SubTest4
sw $3,36($0)
sub $3,$3,$3
ori $3,$3,152
lw $4,-108($3)
beq $3,$4,SubTest4
lw $4,156($0)
sub $0,$0,$0
sw $0,108($0)
sub $4,$3,$2
sub $1,$2,$1
sw $1,20($0)
ori $0,$4,132
sw $1,108($0)
sub $4,$4,$4
sw $1,108($4)
lui $2,80
sub $2,$2,$2
lw $4,16($2)
```

```
add $4,$2,$4
add $3,$0,$0
lw $1,76($0)
sw $0,96($0)
sub $1,$1,$1
lw $4,152($1)
ori $0,$2,68
EndTest4:
jr $ra
sub $3,$4,$2
```

```
Test5:
sub $8,$8,$8
sub $25,$25,$25
ori $25,$25,10
sub $29,$29,$29
ori $29,$29,1
SubTest5:
add $8,$8,$29
beq $8,$25,EndTest5
sub $4,$4,$1
lw $2,96($0)
lui $3,28
lw $3,116($0)
sub $1,$1,$1
ori $1,$1,48
sw $3,-4($1)
add $4,$2,$4
sub $1,$3,$3
sub $2,$2,$2
ori $2,$2,132
lw $4,-112($2)
sub $0,$0,$0
lw $0,148($0)
sub $3,$3,$3
lw $0,12($0)
sub $3,$3,$3
lw $3,136($3)
beq $0,$4,SubTest5
ori $1,$2,0
lui $0,44
beq $1,$2,SubTest5
sub $2,$2,$2
add $1,$0,$0
sw $2,80($0)
lui $0,76
sw $1,88($0)
lui $0,36
sub $1,$2,$2
lw $4,132($0)
beq $0,$3,SubTest5
sw $4,0($0)
sub $2,$2,$2
```

```
ori $2,$2,124
sw $0,-80($2)
sw $2,8($0)
lui $1,0
EndTest5:
jr $ra
lui $0,152
```

```
Test6:
sw $3,136($0)
sub $1,$1,$3
ori $1,$3,0
ori $3,$3,136
lw $2,48($0)
lui $2,40
sw $1,108($0)
lw $0,120($0)
sub $0,$0,$0
lw $2,144($0)
add $1,$4,$1
sub $1,$0,$2
sw $4,124($0)
sub $3,$0,$0
beq $4,$4,EndTest6
ori $2,$2,40
beq $1,$0,EndTest6
sub $1,$3,$1
sub $1,$4,$4
sw $0,52($0)
sub $2,$2,$2
lw $1,24($2)
lui $0,4
ori $1,$0,52
sub $1,$1,$1
ori $1,$1,156
sw $2,-120($1)
lui $1,88
sub $3,$3,$3
sw $1,132($3)
sub $3,$3,$3
sw $0,0($3)
ori $3,$3,136
EndTest6:
jr $ra
ori $4,$1,36
```

```
Test7:
sub $4,$4,$4
sw $4,132($4)
lw $3,56($0)
sub $2,$1,$3
lui $0,48
```

```
sub $4,$4,$4
ori $4,$4,148
sw $1,-112($4)
lw $1,132($0)
lui $4,44
lw $4,140($0)
sub $1,$1,$1
ori $1,$1,128
sw $0,-68($1)
lw $3,116($0)
ori $0,$1,84
sub $3,$3,$3
ori $3,$3,36
lw $3,-24($3)
add $4,$1,$3
sub $3,$3,$3
ori $3,$3,140
sw $1,-136($3)
beq $2,$2,EndTest7
lui $0,152
lui $2,148
beq $4,$0,EndTest7
ori $4,$3,72
beq $3,$2,EndTest7
sw $0,0($0)
sub $4,$4,$4
ori $4,$4,120
lw $2,-4($4)
sw $4,44($0)
lui $1,128
sub $0,$0,$0
sw $0,148($0)
sub $1,$1,$1
sw $3,140($1)
lw $2,128($0)
beq $2,$3,EndTest7
add $3,$4,$4
EndTest7:
jr $ra
sub $1,$0,$0

Test8:
sub $18,$18,$18
sub $23,$23,$23
ori $23,$23,10
sub $17,$17,$17
ori $17,$17,1
SubTest8:
add $18,$18,$17
beq $18,$23,EndTest8
sw $1,8($0)
sub $1,$3,$3
add $0,$2,$3
```



```
sub $3,$3,$3
sw $4,16($3)
sub $4,$4,$4
ori $4,$4,136
sw $3,-128($4)
lw $2,104($0)
sw $3,124($0)
sub $4,$4,$4
ori $4,$4,140
lw $2,-116($4)
sub $4,$4,$4
ori $4,$4,128
sw $3,-24($4)
lui $4,24
sub $3,$3,$3
lw $2,128($3)
sub $2,$2,$2
sw $3,12($2)
sw $1,140($0)
sw $4,72($0)
lw $1,76($0)
beq $3,$1,SubTest8
ori $0,$1,124
sw $3,8($0)
lui $3,4
lui $1,12
beq $3,$4,SubTest8
sw $4,156($0)
add $0,$2,$2
ori $4,$1,60
sub $2,$0,$4
sub $4,$4,$4
ori $4,$4,92
lw $4,-64($4)
lw $3,12($0)
sub $3,$3,$1
EndTest8:
jr $ra
lui $4,24
```

```
Test9:
ori $4,$3,24
add $1,$4,$0
sub $0,$0,$0
lw $1,108($0)
sub $0,$0,$0
sw $1,32($0)
sub $4,$4,$4
lw $4,0($4)
lw $3,84($0)
sub $2,$4,$0
add $1,$3,$1
ori $3,$3,140
```

```
lui $0,52
beq $3,$4,EndTest9
ori $4,$3,76
lw $0,24($0)
lw $3,24($0)
beq $4,$1,EndTest9
sw $4,20($0)
beq $1,$3,EndTest9
lw $4,100($0)
sub $2,$2,$2
ori $2,$2,32
sw $4,0($2)
add $3,$2,$1
sub $3,$3,$3
sw $2,128($3)
ori $2,$4,36
beq $4,$1,EndTest9
lw $0,144($0)
ori $4,$0,100
sub $1,$2,$4
lw $1,140($0)
sub $1,$1,$1
lw $4,148($1)
sub $3,$3,$3
lw $0,56($3)
EndTest9:
jr $ra
lui $4,132
```

```
Test10:
ori $0,$1,60
lui $3,104
sub $3,$4,$1
ori $4,$2,64
sub $1,$1,$1
ori $1,$1,112
lw $3,-32($1)
ori $0,$0,140
sw $1,92($0)
sub $3,$3,$0
ori $2,$2,88
add $4,$0,$3
sw $3,52($0)
add $2,$2,$4
sub $2,$2,$2
ori $2,$2,68
lw $2,-16($2)
sub $3,$3,$3
sw $2,124($3)
ori $4,$3,12
sw $2,152($0)
sub $4,$3,$1
sub $0,$0,$0
```

```
lw $4,8($0)
ori $2,$0,124
sub $4,$4,$4
ori $4,$4,152
sw $4,-104($4)
sub $1,$1,$1
sw $3,144($1)
lui $0,8
sub $4,$4,$4
sw $4,128($4)
beq $3,$2,EndTest10
lui $0,156
sub $3,$0,$4
EndTest10:
jr $ra
sw $2,120($0)
```

```
Test11:
lw $2,44($0)
sub $2,$2,$2
lw $3,0($2)
lui $1,0
ori $2,$0,36
sub $4,$4,$4
lw $3,60($4)
sub $0,$0,$0
sw $1,36($0)
ori $0,$3,56
add $1,$4,$1
beq $3,$4,EndTest11
lw $3,68($0)
sw $2,108($0)
lui $4,16
beq $2,$2,EndTest11
add $2,$4,$4
lui $2,72
sub $2,$3,$4
lw $0,20($0)
sub $1,$1,$1
ori $1,$1,140
lw $4,-16($1)
ori $1,$4,156
lw $0,24($0)
lw $4,40($0)
sub $4,$4,$1
beq $2,$2,EndTest11
sw $1,148($0)
sub $1,$1,$1
ori $1,$1,104
sw $3,-20($1)
sub $4,$4,$4
lw $3,40($4)
ori $1,$2,132
```

```
lui $1,92
EndTest11:
jr $ra
lw $4,4($0)

Test12:
sub $21,$21,$21
sub $29,$29,$29
ori $29,$29,10
sub $17,$17,$17
ori $17,$17,1
SubTest12:
add $21,$21,$17
beq $21,$29,EndTest12
sw $0,152($0)
sw $1,132($0)
sw $1,52($0)
lui $4,100
ori $4,$2,64
add $2,$1,$0
sw $1,124($0)
add $4,$0,$1
beq $3,$2,SubTest12
sw $0,92($0)
lw $3,52($0)
add $0,$2,$2
lui $4,124
lui $4,148
sub $1,$1,$1
ori $1,$1,64
sw $2,-40($1)
sw $2,52($0)
lui $3,156
lui $2,80
sw $1,12($0)
add $4,$0,$1
lui $3,72
sw $4,88($0)
sub $3,$4,$0
sub $2,$2,$2
sw $3,4($2)
lui $0,28
sw $1,72($0)
sub $1,$1,$1
ori $1,$1,84
lw $0,-12($1)
EndTest12:
jr $ra
lui $4,92

Test13:
sub $7,$7,$7
```

```
sub $19,$19,$19
ori $19,$19,10
sub $26,$26,$26
ori $26,$26,1
SubTest13:
add $7,$7,$26
beq $7,$19,EndTest13
sub $4,$0,$1
sub $2,$2,$2
lw $3,24($2)
sub $1,$1,$1
sw $2,20($1)
lw $0,4($0)
sw $4,120($0)
lw $3,124($0)
lui $2,28
sub $4,$4,$4
ori $4,$4,136
lw $3,-80($4)
add $3,$1,$4
ori $1,$2,152
sub $4,$4,$4
ori $4,$4,128
lw $3,-72($4)
lui $0,20
beq $3,$2,SubTest13
sw $1,24($0)
sub $4,$4,$4
sw $3,136($4)
sub $4,$4,$4
sw $1,136($4)
sub $2,$2,$2
lw $1,120($2)
sw $1,88($0)
sub $4,$0,$0
sub $2,$2,$2
ori $2,$2,88
sw $3,-80($2)
sw $1,148($0)
lui $4,84
add $0,$1,$0
lui $0,64
sub $1,$1,$1
sw $4,80($1)
ori $3,$3,132
sub $3,$3,$3
lw $0,112($3)
EndTest13:
jr $ra
lui $0,148
```

```
Test14:
sub $24,$24,$24
```

```
sub $5,$5,$5
ori $5,$5,10
sub $19,$19,$19
ori $19,$19,1
SubTest14:
add $24,$24,$19
beq $24,$5,EndTest14
lw $3,48($0)
ori $4,$1,60
sw $2,20($0)
lw $3,64($0)
lui $0,48
add $2,$0,$0
lw $2,36($0)
add $1,$0,$2
ori $0,$4,104
beq $3,$4,SubTest14
add $4,$0,$2
lw $2,148($0)
sub $1,$1,$1
ori $1,$1,88
lw $1,-80($1)
sub $1,$1,$1
ori $1,$1,148
lw $4,-16($1)
sub $0,$2,$2
sub $2,$2,$2
ori $2,$2,80
sw $0,-72($2)
beq $4,$2,SubTest14
sw $0,124($0)
sub $0,$0,$0
lw $0,124($0)
add $4,$4,$4
sw $1,44($0)
sub $3,$2,$4
add $3,$1,$4
sub $4,$4,$4
sw $4,104($4)
sw $1,16($0)
beq $4,$3,SubTest14
sw $1,112($0)
sub $1,$1,$1
ori $1,$1,48
lw $4,0($1)
beq $1,$2,SubTest14
sub $0,$3,$0
EndTest14:
jr $ra
sw $3,40($0)

Test15:
sub $15,$15,$15
```

```
sub $29,$29,$29
ori $29,$29,10
sub $6,$6,$6
ori $6,$6,1
SubTest15:
add $15,$15,$6
beq $15,$29,EndTest15
add $0,$2,$1
lui $1,96
sub $4,$2,$2
sub $1,$2,$1
ori $4,$4,32
sw $1,20($0)
add $1,$3,$2
sub $0,$0,$0
sw $0,128($0)
sw $2,104($0)
add $0,$1,$2
beq $2,$3,SubTest15
lw $4,32($0)
lw $1,48($0)
sw $3,108($0)
sub $4,$0,$4
sub $3,$3,$3
ori $3,$3,144
lw $4,-76($3)
ori $3,$1,80
sub $1,$2,$4
sw $4,36($0)
lui $1,100
beq $3,$0,SubTest15
lui $2,140
add $0,$0,$2
sub $3,$3,$3
ori $3,$3,92
lw $0,-80($3)
lui $2,76
lui $3,12
lw $4,56($0)
ori $2,$3,36
EndTest15:
jr $ra
sub $3,$4,$2
End:
```